Retrieval-Augmented Generation (RAG): Document Loader

Definition:

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with language generation, where a model first retrieves relevant documents from a knowledge base and then uses them as context to generate accurate, relevant, and grounded responses.

Benefits of Using RAG

1. Up-to-date Information:

RAG can fetch the latest data at query time instead of relying only on what the model was trained on.

Real-world example:

- In a **stock market assistant**, instead of giving outdated prices, RAG retrieves **live stock data** before answering.
- Query: *"What is the current price of TCS stock?"*
- RAG fetches latest data → gives accurate answer

Why LLM alone cannot do this:

- LLMs are **static** (trained on past data with a cutoff date)
- No built-in access to **real-time APIs or databases**
- Would either:
    - Give outdated info
    - Or hallucinate a price

2. Improved Accuracy & Reduced Hallucination

The model answers based on retrieved documents, not assumptions.

Real-world example:

- In a **medical chatbot**, RAG retrieves **verified clinical documents**
- Query: *"Treatment for dengue?"*
- Response is grounded in real guidelines

Why LLM alone cannot do this:

- LLM generates answers based on **probability, not truth**
- No guarantee the answer is:
    - factual

    - source-backed
- High risk in domains like **medical, legal, finance**

3. Better Data Privacy

RAG enables usage of **private/internal data** securely.

Real-world example:

- In a **network monitoring system (ELK + logs)**
- Query: *"Why did packet drops increase at 10 PM?"*
- RAG retrieves internal logs → explains issue

Why LLM alone cannot do this:

- LLM does **not have access to your internal systems**
- Cannot:
    - read Elasticsearch logs
    - access private DBs
- Also:
    - You **cannot train LLM every time** with sensitive data

4. No Document Size Limitation

RAG retrieves only relevant chunks instead of processing entire data.

Real-world example:

- In a **legal assistant** with thousands of pages
- Query: *"What are termination clauses?"*
- RAG extracts only relevant sections

Why LLM alone cannot do this:

- LLM has **context window limits** (even GPT has limits)
- Cannot:
    - process huge documents fully
    - efficiently search inside them
- Would either:
    - truncate data
    - miss important context

5. Cost Efficient
No need to retrain models for new data.

Real-world example:

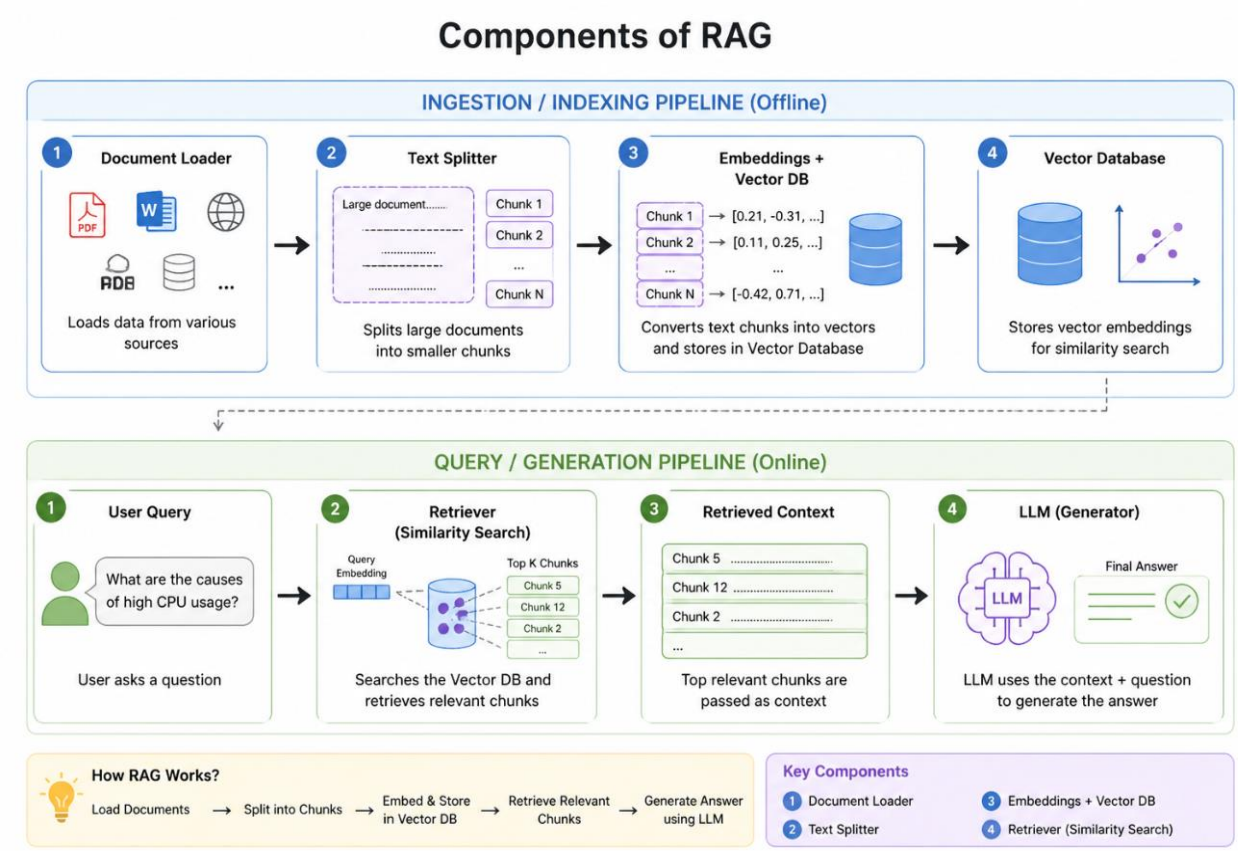
- In a **customer support chatbot**
- FAQs update daily
- RAG just retrieves new data dynamically

Why LLM alone cannot do this:

- Requires **frequent fine-tuning** for updates
- Fine-tuning is:
    - expensive
    - slow
- Not scalable for dynamic data

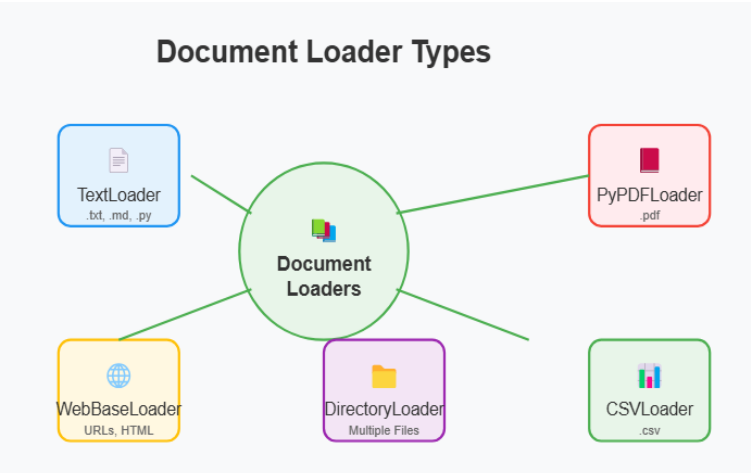
Components of RAG:

1. Document Loader
2. Text Splitter
3. Vector Database
4. Retriever



1. Document Loader:

Document loaders are tools that play a crucial role in data ingestion. They take in raw data from different sources and convert them into a structured format called “Documents”. These documents contain the document content as well as the associated metadata like source and timestamps. Let’s look into the different types of document loaders.



Document Loaders are classes that extract raw data from various sources and convert them into standardized Document objects, which include below.

- **page_content:** The actual text data.
- **metadata:** Info like source, URL, title, or timestamp.
- **id**: Uniquely identifies the document object

Common Langchain Document Loader:

Types of Document Loaders

LangChain provides over 200 document loaders, categorized as follows:

- **By File Type:** CSV, PDF, HTML, Markdown, MS Office documents, JSON.
- **By Data Source:** YouTube, Wikipedia, GitHub.

1. **Text Loader:**
   TextLoader class is used to load text files with plain text. These files have .txt extension. Below is an example of how to load a text file using TextLoader

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("AI_Introduction.txt")
documents = loader.load()

print(f"Type of Documents: {type(documents)}")
print(f"Type of Document in Documents: {type(documents[0])}")
print(f"Number of Documents: {len(documents)}")

print(f"Actual Data:\n{documents[0].page_content}\n")
print(f"Meta Data:\n{documents[0].metadata}\n")
```

2. **PyPDF Loader: limitations and solutions with other pdf loader**

This is used to load content from .pdf files. There are different classes of pdf loader depending on the data present in the file.

- Simple, clean PDFs: Use PyPDFLoader
- PDFs with tables/columns: Use PDFPlumberLoader

- Scanned/image PDFs: Use UnstructuredPDFLoader or AmazonTextractPDFLoader
- Need layout and image data: Use PyMuPDFLoader
- Want best structure extraction: Use UnstructuredPDFLoader

We will use PyPDFLoader in the example below.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("AI_Introduction.pdf")
documents = loader.load()

print(f"Actual Data:\n{documents[0].page_content}\n")
print(f"Meta Data:\n{documents[0].metadata}\n")
```

3. Directory Loader (load vs lazy load):

Directory loader helps in loading data from multiple files stored in a specific directory.

DirectoryLoader loads files from a folder using UnstructuredLoader by default, which supports formats like PDF, HTML, and Markdown. We can use the glob parameter to include specific file types—e.g., load only .pdf files while skipping .csv and .txt.

```
from langchain_community.document_loaders import DirectoryLoader
from langchain_community.document_loaders import PyPDFLoader

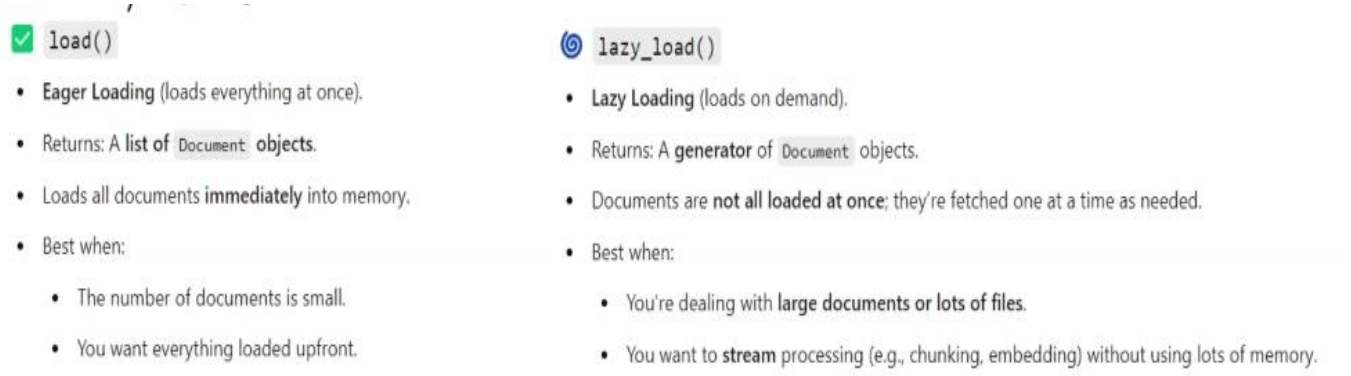
loader = DirectoryLoader("Data Folder/", glob="**/*.pdf", loader_cls=PyPDFLoader,
    show_progress=True)
documents = loader.load()

print(f"Number of Documents: {len(documents)}")

for idx, document in enumerate(documents, start=1):
    print(f"\nDocument {idx}")
    print("Content:\n", document.page_content)
    print("Metadata:\n", document.metadata)
```

- load() → loads all documents at once (eager loading)

- lazy_load() → loads documents one by one as you iterate (lazy loading)



4. WebBase Loader :

Web Page Loader is used to load content from the website. It uses **requests** and **BeautifulSoup4** python libraries inthe backend. We simply need to pass the URL of the website.
In the example below, I am loading this website:
https://medium.com/@mangeshsalunke1309/list/generative-ai-langchain-fca1591f6e98

```
from langchain_community.document_loaders import WebBaseLoader

URL = "https://medium.com/@mangeshsalunke1309/list/generative-ai-langchain-fca1591f6e98"

loader = WebBaseLoader(URL)
documents = loader.load()

# print(documents)
print(f"Actual Data:\n{documents[0].page_content}")
print(f"Meta Data:\n{documents[0].metadata}\n")
```

5. CSV Loader

In LangChain, the CSVLoader is the primary tool used to load data from CSV files and transform each row into a standardized Document object.

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader(file_path='./your_data.csv')
data = loader.load()
```

Project Idea: make a chrome plugin ... whenever we open any web page the plugin will help to know about the document with chat.